

Despliegue Productivo: Terraform + Docker + App Service + GitHub Actions

Flujo final

1. `infra-terraform.yml` valida y aplica IaC cuando cambian archivos de infraestructura.
2. `app-container-deploy.yml` construye imagen Docker, la sube a ACR y actualiza `docker_image_tag` via Terraform.
3. App Service hace pull de la imagen desde ACR usando Managed Identity + rol `AcrPull`.
4. El backend arranca en puerto `8080` y se valida con `/health`.

Variables de app que quedan alineadas al código

La app usa:

- `DB_HOST`
- `DB_PORT`
- `DB_NAME`
- `DB_USER`
- `DB_PASSWORD`
- `JWT_SECRET`
- `JWT_EXPIRES_IN`
- `SMTP_HOST`
- `SMTP_PORT`
- `SMTP_USER`
- `SMTP_PASS`
- `FRONTEND_URL`

Estas variables se configuran desde Terraform en `azurerm_linux_web_app.app_settings`.

Secretos requeridos en GitHub (Environment: prod)

- `AZURE_CLIENT_ID`
- `AZURE_TENANT_ID`
- `AZURE_SUBSCRIPTION_ID`
- `ACR_NAME`
- `ACR_LOGIN_SERVER`

Recomendaciones de hardening para producción

- Habilitar backend remoto de Terraform (`backend "azurerm"`).
- Separar `dev`, `staging`, `prod` con tfvars y ambientes.
- Usar `S1/P1v3` o superior para App Service productivo.
- Evitar firewall `0.0.0.0` en PostgreSQL cuando tengas VNet/Private Endpoint.
- Guardar secretos sensibles en Key Vault y no en tfvars.

- Agregar deployment slots para zero-downtime.

Primer despliegue sugerido

1. Completar `infra/terraform.tfvars` con valores reales.
2. Ejecutar import de infraestructura existente (`infra/IMPORT_EXISTING_INFRA.md`).
3. Confirmar `terraform plan` sin cambios destructivos.
4. Hacer merge a `main` para disparar workflows.